



Reading Assignment 2

Report

- **College:** College of Computing
- **Major:** Computer Science
- **Student Name:** CHEN Xian
- **Student Number:** 59790558

Feb. 13th 2026

Practical Byzantine Fault Tolerance (PBFT)

1 Initial Challenges

- The necessity of the three-phase protocol: It was initially unclear why pre-prepare, prepare, and commit phases were all strictly required, rather than just a two-step proposal and acceptance.
- The theoretical bound of replicas: The mathematical justification for why exactly $|\mathcal{R}| = 3f + 1$ replicas are needed to tolerate f faults in an asynchronous system was difficult to intuitively grasp.
- View-change mechanics: The process of a primary failing and the subsequent view change seemed overly complex, specifically how the new primary guarantees that no committed requests from the old view are lost.

2 Process of Inquiry

Interaction 1: Deciphering the Three-Phase Protocol

My Prompt: "Why does PBFT strictly require a 'commit' phase? Wouldn't the 'pre-prepare' and 'prepare' phases be enough to establish a total order of requests?"

My Analysis: The AI's response was highly illuminating. It clarified that while the pre-prepare and prepare phases successfully guarantee that non-faulty replicas agree on a total order for requests within a specific view. However, they do not protect against view changes. The AI pointed out that the commit phase is essential to ensure that requests that commit are totally ordered across different views. If a primary fails and a view change occurs, the commit phase ensures enough replicas have synchronized states to prove the sequence to the new primary. This generated a new question for me regarding how the new primary handles gaps in the sequence.

Interaction 2: Handling Sequence Gaps during View Changes

My Prompt: "During a view change, if the new primary finds a gap in the sequence numbers of prepared requests, how does it resolve this without stalling the system?"

My Analysis: The AI accurately referenced the paper's mechanism for handling gaps, explaining that the new primary creates a new pre-prepare message containing a special null request for the missing sequence number. The execution of this null request acts as a no-op. I cross-referenced this with the paper and confirmed it is correct. This prevents a malicious old primary from intentionally creating unresolvable gaps to halt system progress.

3 Synthesis and Lingering Questions

My overall understanding of PBFT has shifted from viewing it as a simple voting mechanism to recognizing it as a highly robust, state-machine replication algorithm built on cryptographic proofs. The core mechanics rely on heavily redundant messaging to bypass the impossibility of consensus in purely asynchronous networks. The pragmatic use of message authentication

codes (MACs) during normal operations, rather than expensive public-key signatures, is the true catalyst for its practical performance.

The protocol requires each backup to multicast a prepare message to all other replicas, and then all replicas multicast a commit message to all other replicas. This creates a massive communication overhead as the number of nodes increases. How could this be adapted for large-scale, permissionless environments without degrading performance?